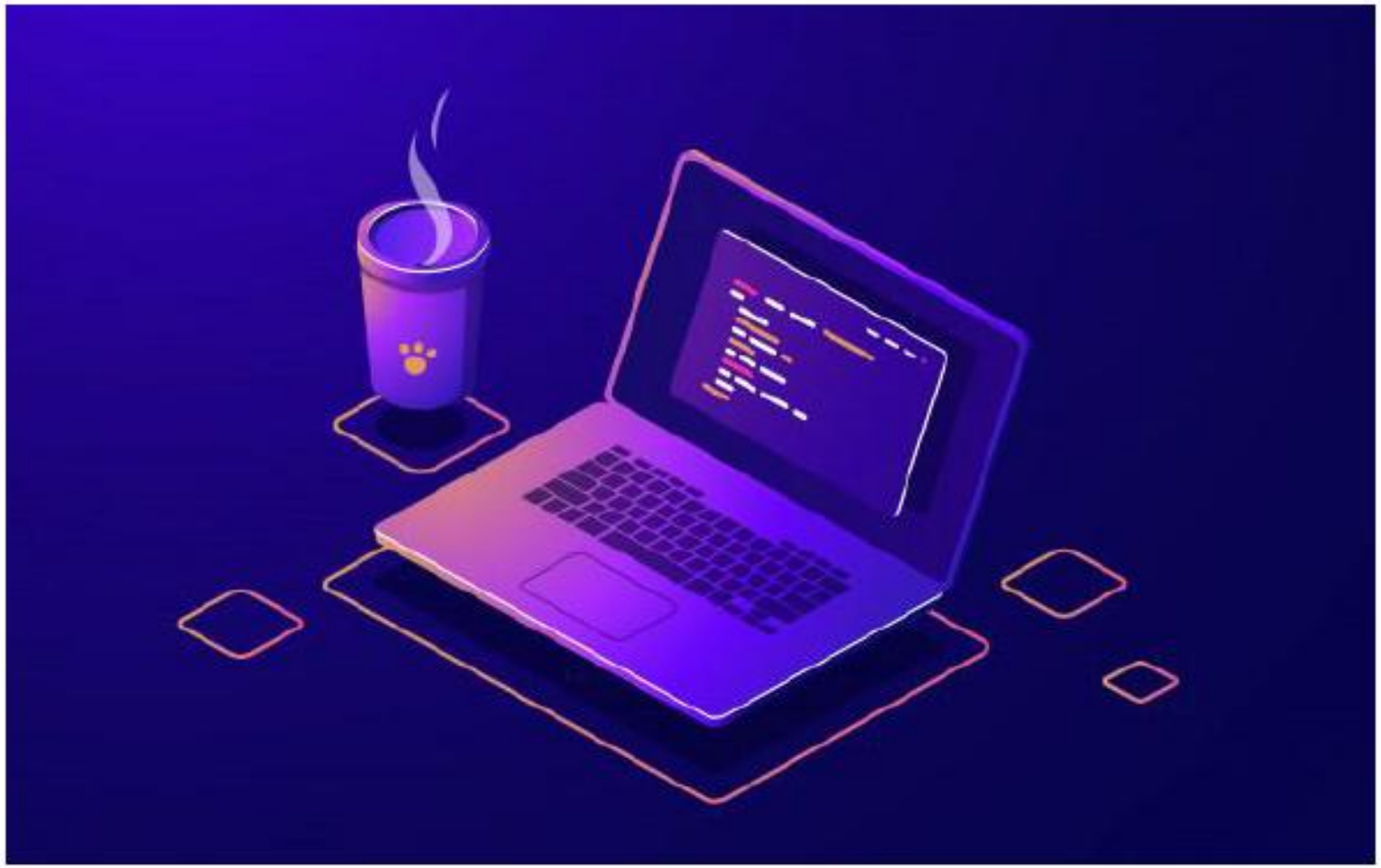


PROGRAMACIÓN

TEMA 10: GESTIÓN DE BASES DE DATOS RELACIONALES

GESTIÓN DE BBDD RELACIONALES

ÍNDICE

10.1 INTRODUCCIÓN

10.1.1 El desfase objeto-relacional

10.1.2 JDBC

10.1.3 Conectores o Drivers.

10.2 CREACIÓN DE LAS TABLAS EN UNA BASE DE DATOS

10.2.1 Creación de tablas con MySQL

10.2.2 Lenguaje SQL

10.2.3 Establecimiento de conexiones

10.2.4 Instalar el conector de la base de datos

10.2.5 Registrar el controlador JDBC

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BASE DE DATOS

10.3.1 Recuperación de información

10.3.2 Actualización de información

10.3.3 Adición de información

10.3.4 Borrado de información

10.3.5 Cierre de conexiones

10.3.6 Excepciones

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

Hoy en día, la mayoría de aplicaciones informáticas necesitan almacenar y gestionar gran cantidad de datos.

Esos datos, se suelen guardar en bases de datos relacionales, ya que éstas son las más extendidas actualmente.

Las bases de datos relacionales permiten organizar los datos en tablas y esas tablas y datos se relacionan mediante campos clave. Además, se trabaja con el lenguaje estándar conocido como SQL, para poder realizar las consultas que deseemos a la base de datos.

Una base de datos relacional se puede definir de una manera simple como aquella que presenta la información en tablas con filas y columnas.

Una tabla es una serie de filas y columnas, en la que cada fila es un ***registro*** y cada columna es un ***campo***. Un campo representa un dato de los elementos almacenados en la tabla (NSS, nombre, etc.)

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

No se permite que pueda aparecer dos o más veces el mismo registro, por lo que uno o más campos de la tabla forman lo que se conoce como clave primaria (atributo que se elige como identificador en una tabla, de manera que no haya dos registros iguales, sino que se diferencien al menos en esa clave.

Por ejemplo, en el caso de una tabla que guarda datos de personas, el número de la seguridad social, podría elegirse como clave primaria[4], pues sabemos que aunque haya dos personas llamadas, por ejemplo, Juan Pérez Pérez, estamos seguros de que su número de seguridad social será distinto).

El sistema gestor de bases de datos, en inglés conocido como: Database Management System (**DBMS**), gestiona el modo en que los datos se almacenan, mantienen y recuperan.

En el caso de una base de datos relacional, el sistema gestor de base de datos se denomina: Relational Database Management System (**RDBMS**).

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

Tradicionalmente, la programación de bases de datos ha sido como una Torre de Babel: gran cantidad de productos de bases de datos en el mercado, y cada uno "hablando" en su lenguaje privado con las aplicaciones.

Java, mediante **JDBC (Java Database Connectivity**. API que permite la ejecución de operaciones sobre bases de datos desde el lenguaje de programación Java, independientemente del sistema operativo donde se ejecute o de la base de datos a la cual se accede), permite simplificar el acceso a base de datos, proporcionando un lenguaje mediante el cual las aplicaciones pueden comunicarse con motores de bases de datos. SUN desarrolló este API para el acceso a bases de datos, con tres objetivos principales en mente:

- ✓ Ser un API con soporte de SQL: poder construir sentencias SQL e insertarlas dentro de llamadas al API de Java
- ✓ Aprovechar la experiencia de los APIs de bases de datos existentes,
- ✓ Ser sencillo.

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

10.1.1 DESFASE OBJETO-RELACIONAL

El desfase objeto-relacional, también conocido como impedancia objeto-relacional, consiste en la diferencia de aspectos que existen entre la programación orientada a objetos y la base de datos. Estos aspectos se pueden presentar en cuestiones como:

- **Lenguaje de programación**. El programador debe conocer el lenguaje de programación orientada a objetos (POO) y el lenguaje de acceso a datos.
- **Tipos de datos**: en las bases de datos relacionales siempre hay restricciones en cuanto a los tipos de datos que se pueden usar, que suelen ser sencillos, mientras que la programación orientada a objetos utiliza tipos de datos más complejos.
- **Paradigma de programación**. En el proceso de diseño y construcción del software se tiene que hacer una traducción del modelo orientado a objetos de clases al modelo Entidad-Relación (E/R) puesto que el primero maneja objetos y el segundo maneja tablas y tuplas o filas, lo que implica que se tengan que diseñar dos diagramas diferentes para el diseño de la aplicación.

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

10.1.1 DESFASE OBJETO-RELACIONAL

El modelo relacional trata con relaciones y conjuntos debido a su **naturaleza matemática**. Sin embargo, el modelo de Programación Orientada a Objetos trata con objetos y las asociaciones entre ellos. Por esta razón, el problema entre estos dos modelos surge en el momento de querer persistir los objetos de negocio

La escritura (y de manera similar la lectura) mediante **JDBC** implica: **abrir** una conexión, **crear** una sentencia en SQL y **copiar** todos los valores de las propiedades de un objeto en la sentencia, **ejecutarla** y así **almacenar** el objeto.

Esto es sencillo para un caso simple, pero trabajoso si el objeto posee muchas propiedades, o bien se necesita almacenar un objeto que a su vez posee una colección de otros elementos. Se necesita crear mucho más código, además del tedioso trabajo de creación de sentencias SQL.

Este problema es lo que denominábamos **impedancia Objeto-Relacional**, o sea, el conjunto de dificultades técnicas que surgen cuando una base de datos relacional se usa en conjunto con un programa escrito en con u lenguajes de Programación Orientada a Objetos.

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

10.1.2 JDBC

JDBC es un API Java que hace posible ejecutar sentencias SQL.

De JDBC podemos decir que:

- Consta de un **conjunto de clases e interfaces** escritas en Java.
- Proporciona un API estándar para desarrollar aplicaciones de bases de datos con un API Java pura.

Con JDBC, no hay que escribir un programa para acceder a una base de datos Access, otro programa distinto para acceder a una base de datos Oracle, etc., sino que **podemos escribir un único programa** con el API JDBC y el programa se encargará de enviar las sentencias SQL a la base de datos apropiada. Además, y como ya sabemos, una aplicación en Java puede ejecutarse en plataformas distintas.

En el desarrollo de JDBC, y debido a la confusión que hubo por la proliferación de API's propietarios de acceso a datos, Sun buscó los aspectos de éxito de un API de este tipo, ODBC.

GESTIÓN DE BBDD RELACIONALES

10.1 NTRODUCCIÓN

10.1.2 JDBC

ODBC se desarrolló con la idea de tener un estándar para el acceso a bases de datos en entorno Windows.

Aunque la industria ha aceptado ODBC como medio principal para acceso a bases de datos en Windows, ODBC no se introduce bien en el mundo Java, debido a la complejidad que presenta ODBC, y que entre otras cosas ha impedido su transición fuera del entorno Windows.

El nivel de abstracción al que trabaja JDBC es alto en comparación con ODBC, la intención de Sun fue que supusiera la base de partida para crear librerías de más alto nivel.

JDBC intenta ser tan simple como sea posible, pero proporcionando a los desarrolladores la máxima flexibilidad.

GESTIÓN DE BBDD RELACIONALES

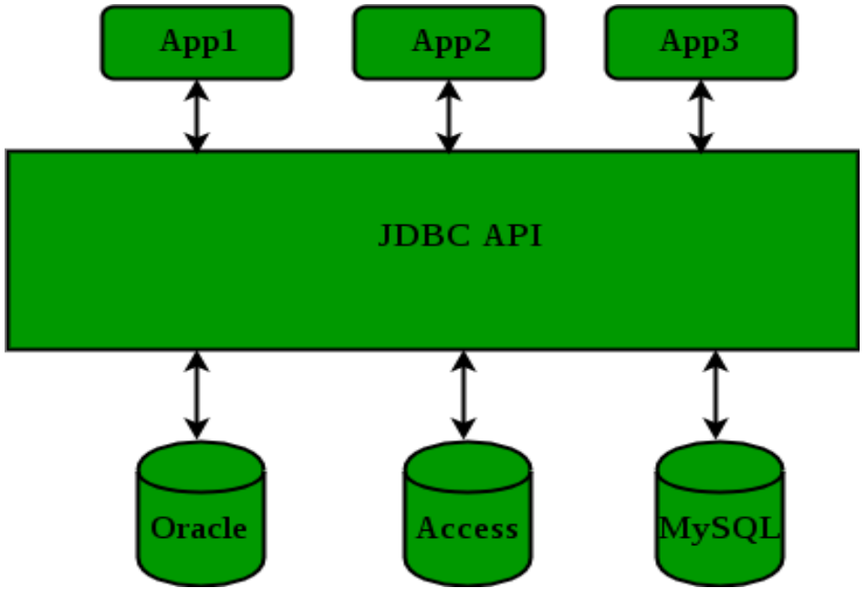
10.1 INTRODUCCIÓN

10.1.3 CONECTORES O DRIVERS

El API JDBC viene distribuido en dos paquetes:

- java.sql, dentro de J2SE
- javax.sql, extensión dentro de J2EE

Un conector o driver es un conjunto de clases encargadas de implementar las interfaces del API y acceder a la base de datos.



GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

10.1.3 CONECTORES O DRIVERS

Para poder conectarse a una base de datos y lanzar consultas, una aplicación necesita tener un conector adecuado. Un conector suele ser un fichero .jar que contiene una implementación de todas las interfaces del API JDBC.

Cuando se construye una aplicación de base de datos, **JDBC oculta los detalles específicos de cada base de datos**, de modo que le programador se ocupe sólo de su aplicación.

El conector lo proporciona el fabricante de la base de datos o bien un tercero.

El código de nuestra aplicación no depende del driver, puesto que trabajamos contra los paquetes java.sql y javax.sql.

JDBC ofrece las clases e interfaces para:

- Establecer una conexión a una base de datos.
- Ejecutar una consulta.
- Procesar los resultados.

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

10.1.3 CONECTORES O DRIVERS

```
// Establece la conexión
Connection con = DriverManager.getConnection ("jdbc:odbc:miBD", "miLogin", "miPassword");

// Ejecuta la consulta
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT nombre, edad FROM Jugadores");

// Procesa los resultados
while (rs.next()) {
    String nombre = rs.getString("nombre");
    int edad = rs.getInt("edad");
}
```

En principio, todos los conectores[7] deben ser compatibles con ANSI SQL- 2 Entry Level (ANSI SQL-2 se refiere a los estándares adoptados por el **American National Standards Institute** (ANSI) en 1992.

Entry Level se refiere a una lista específica de capacidades de SQL).

Los desarrolladores de conectores pueden establecer que sus conectores conocen estos estándares.

GESTIÓN DE BBDD RELACIONALES

10.1 INTRODUCCIÓN

10.1.3 CONECTORES O DRIVERS

(Clasificación clásica)

- **Tipo 1**: puente JDBC–ODBC (obsoleto)
- **Tipo 2**: API nativa + JNI (dependiente de SO)
- **Tipo 3**: middleware (raro hoy)
- **Tipo 4**: “pure Java”, habla directo con el motor (**el habitual**)

Errores típicos:

- **“No suitable driver”** → falta dependencia / classpath incorrecto
- Errores de autenticación → usuario/clave/permisos
- Errores de URL → host/puerto/bd/parámetros
- Objetivo práctico: saber identificar el problema por el mensaje y el SQLState.

GESTIÓN DE BBDD RELACIONALES

INTRODUCCIÓN

CONECTORES O DRIVERS. Establecimiento de Conexiones.

Cuando queremos acceder a una base de datos para operar con ella, lo primero que hay que hacer es conectarse a dicha base de datos.

En Java, para establecer una conexión con una base de datos podemos utilizar el método *getConnection()* de la clase *DriverManager*. Este método recibe como parámetro la URL de JDBC que identifica a la base de datos con la que queremos realizar la conexión.

```
publica static void main(String[] args) {
    String url = "jdbc:mysql://localhost:3306/tu_base_de_datos";
    // Cambia a tu configuración
    String user = "tu_usuario";
    String password = "tu_contraseña";
    try ( Connection connection = DriverManager.getConnection(url, user, password)) {
       System.out.println("¡Conexión exitosa!");
    } catch (SQLException e) {
      e.printStackTrace();
    }
}
```

GESTIÓN DE BBDD RELACIONALES

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        // Datos de conexión
        String url = "jdbc:mysql:
        //localhost:3306/tu_base_de_datos";
       // Cambia el nombre de la base de datos
        String user = "tu_usuario";
       // Usuario de la base de datos
        String password = "tu_contraseña";
       // Contraseña del usuario
        Connection connection = null;
        try {
                // Establecer la conexión
                connection = DriverManager.getConnection(url, user, password);
                System.out.println("¡Conexión a la base de datos exitosa!");
            } catch (SQLException e) {
                // Manejo de errores
                System.err.println("Error al conectar con la base de datos:");
                e.printStackTrace();

            } finally {
                        if (connection != null) {
                            try {
                                connection.close();
                                // Cerrar la conexión
                                System.out.println("Conexión cerrada.");
                            } catch (SQLException e) {
                                System.err.println("Error al cerrar la conexión:");
                                e.printStackTrace();
                            }}}}}
```

GESTIÓN DE BBDD RELACIONALES

INTRODUCCIÓN

CONECTORES O DRIVERS.

Instalar el Conector.

Descargar el controlador JDBC

1. **Identifica el controlador JDBC que necesitas:**
   - Si estás usando **MySQL**, descarga el controlador JDBC de MySQL Connector/J.
   - Para **PostgreSQL**, descárgalo de PostgreSQL JDBC.
   - Para otras bases de datos, busca el controlador JDBC correspondiente.
2. Descarga el archivo .jar del controlador desde el sitio oficial.

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

- Se asume BD y tablas creadas (asignatura de BBDD).
- Aquí solo necesitas conocer:
  - nombre de tabla y columnas
  - tipos relevantes (texto, entero, decimal, fecha…)
  - PK (para WHERE id=?) y relaciones (para consultas)
  - El peso real de 10.2 en este tema:
    - **10.2.3 Conexión**
    - **10.2.4 Dependencia/driver**
    - **10.2.5 Registro/descubrimiento del driver**

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

reparar la BD para que la aplicación pueda trabajar:

- Crear BD y usuario (si aplica)
- Crear tablas (DDL)
- Definir claves e integridad
- Dejar lista la conectividad (driver + conexión)

10.2.1 Creación de BBDD con MySQL

- En esta asignatura se asume:
  - BD y tablas **ya existen** (creadas en BBDD).
- Aquí solo interesa saber:
  - **qué columnas hay** (para mapear `ResultSet` a objetos)
  - **qué PK/FK** existen (para `JOIN`, borrados, integridad)
- Si necesitas un ejemplo de esquema para practicar JDBC (muy simple):
  - `alumnos(id PK, nombre, edad)`
  - (sin profundizar en DDL)

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.1–10.2.2 (mínimo): SQL que “se usa” desde Java

- No se explica SQL como lenguaje; se usa como “comando” a ejecutar desde Java.
- Tipos de sentencias que verás en código:
  - `SELECT ... WHERE ...`
  - `INSERT ... VALUES (...)`
  - `UPDATE ... SET ... WHERE ...`
  - `DELETE ... WHERE ...`
- Regla clave para JDBC:
  - **no concatenar datos** dentro del SQL
  - usar **`?`** + **`PreparedStatement`**

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.3 URL JDBC y Establecimiento de Conexiones.

- Formato típico:
  - `jdbc:mysql://HOST:PUERTO/BD?parametros`
- Parámetros habituales (según entorno):
  - `serverTimezone=UTC`
  - `useSSL=false` (laboratorio local)
  - `characterEncoding=UTF-8` (si procede)
- Ejemplo:

```
String url = "jdbc:mysql://localhost:3306/centro?serverTimezone=UTC";
```

- En este caso:
  - La URL se conecta al servidor (localhost:3306), no a una base de datos específica.
  - La sentencia SQL CREATE DATABASE miNuevaBaseDeDatos crea una base de datos.

- Después de crear la base de datos, puedes conectarte a ella modificando la URL de conexión.

```
String url = "jdbc:mysql://localhost:3306/miNuevaBaseDeDatos";
try (Connection conn = DriverManager.getConnection(url, "root", "1234")) {
    System.out.println("Conectado a la nueva base de datos.");
}
```

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.3 Abrir Conexión desde Java.

```
String url = "jdbc:mysql://localhost:3306/centro?serverTimezone=UTC";
String user = "app";
String pass = "app123";

try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    System.out.println("Conexion OK");
}
```

- Connection representa una sesión con la BD.
- Por defecto suele estar autoCommit=true (importa en transacciones).
- No dejar conexiones abiertas: saturas servidor/pool y bloqueas recursos.
  - try-with-resources cierra la conexión automáticamente.
- Errores típicos aquí:
  - URL mal formada
  - servidor apagado/puerto incorrecto
  - usuario/clave/permisos

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.4 Instalar el Conector.

Maven

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

Gradle

```
implementation "com.mysql:mysql-connector-j:8.4.0"
```

Manual (IDE):

- añadir `.jar` al classpath del proyecto

Síntoma de ausencia:

- `SQLException: No suitable driver found for jdbc:mysql://...`

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.5 Registrar el Conector en la BBDD.

- JDBC moderno (Java 6+, JDBC 4+): el driver suele **autoregistrarse** si está en classpath.
- Modo clásico (a veces exigido):

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- Diagnóstico:
  - falla `Class.forName` → dependencia/classpath incorrecto
  - falla `getConnection` → conexión/credenciales/servidor

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.5 Registrar el Conector en la BBDD. En IntelliJ Idea

Proyecto sin gestor (añadir el .jar manualmente)

1. Descarga el `.jar` del driver (p.ej. `mysql-connector-j-8.x.x.jar`).
2. En IntelliJ:
    1. *File → Project Structure…*
    2. **Modules → Dependencies**
    3. Pulsa + → **JARs or directories…**
    4. Selecciona el `.jar`
    5. Asegúrate de que el “Scope” sea **Compile** (por defecto suele estar bien)
3. Aplica cambios (**Apply / OK**).

Resultado: el jar queda en el classpath del módulo.

GESTIÓN DE BBDD RELACIONALES

10.2 CREACIÓN DE LAS TABLAS EN UN BD

10.2.5 Registrar el Conector en la BBDD. En IntelliJ Idea

10.2.5.1 Checklist de “conexión lista”

- Driver instalado correctamente
- URL correcta (host/puerto/bd/parámetros)
- Credenciales válidas
- Código de prueba mínimo:
  - abrir conexión y cerrar (sin consultas)
- Si falla: anotar `getMessage()`, `SQLState`, `ErrorCode` (ver 10.3.6)

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

API Mínima para ejecutar SQL.

- Clases JDBC que usarás siempre:
  - Connection: canal hacia BD
  - Statement / PreparedStatement: ejecutar SQL
  - ResultSet: leer resultados de SELECT
- Dos métodos críticos:
  - executeQuery() → SELECT
  - executeUpdate() → INSERT/UPDATE/DELETE

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

Statement y PreparedStatement (explícito y evaluable).

`Statement`

- Ejecuta SQL como texto completo.
- Se usa así: `st.executeQuery(sql)` / `st.executeUpdate(sql)`
- Riesgo si concatenas variables: **inyección SQL**.

`PreparedStatement` (recomendado)

- SQL “preparado” con `?`
- Se usa así: `ps.setX(pos, valor)` y luego `ps.executeQuery()` / `ps.executeUpdate()`
- Ventajas: seguridad, tipado, robustez, posible mejora de rendimiento.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

Statement y PreparedStatement (explícito y evaluable).

Uso de Statement: SELECT

```
String sql = "SELECT id, nombre, edad FROM alumnos ORDER BY nombre";

try (Statement st = conn.createStatement();
     ResultSet rs = st.executeQuery(sql)) {

    while (rs.next()) {
        int id = rs.getInt("id");
        String nombre = rs.getString("nombre");
        int edad = rs.getInt("edad");
    }
}
```

- Patrón correcto de cierre: `Statement` + `ResultSet` en try-with-resources.
- Aun así: para consultas con filtros variables, mejor `PreparedStatement`.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

Statement y PreparedStatement (explícito y evaluable).

Uso de Statement: UPDATE / INSERT / DELETE

```
String sql = "UPDATE alumnos SET edad = 20 WHERE id = 7";

try (Statement st = conn.createStatement()) {
    int filas = st.executeUpdate(sql);
    // filas afectadas
}
```

- executeUpdate(sql) devuelve número de filas afectadas.
- Si filas == 0: condición no coincidió (p.ej., id inexistente).

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

Statement y PreparedStatement (explícito y evaluable).

Problema del Statement:

concatenación e inyección (no hacerlo)

```
String nombre = inputUsuario;
String sql = "SELECT id FROM alumnos WHERE nombre = '" + nombre + "'";
```

- Si `nombre` contiene comillas o SQL, puede:
  - romper la consulta
  - manipular la consulta (inyección)
- Conclusión: `Statement` solo para SQL fijo/controlado.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

Statement y PreparedStatement (explícito y evaluable).

Uso de PreparedStatement: patrón base (SELECT)

```
String sql = "SELECT id, nombre, edad FROM alumnos WHERE edad >= ? ORDER BY nombre";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 18);

    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            int id = rs.getInt("id");
            String nombre = rs.getString("nombre");
            int edad = rs.getInt("edad");
        }
    }
}
```

- Ventaja: el motor recibe SQL + parámetros separados.
- No hay que gestionar comillas manualmente.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

Statement y PreparedStatement (explícito y evaluable).

Uso de PreparedStatement: Parámetros tipados y nulos.

- Asignación tipada:
  - `setInt,setString,setBigDecimal,setDate,setTimestamp…`
- Insertar NULL:

```
ps.setNull(1, java.sql.Types.VARCHAR);
```

- Leer posibles NULL:

```
int edad = rs.getInt("edad");
if (rs.wasNull()) {
    // era NULL en BD
}
```

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.1 Recuperación: executeQuery() + ResultSet (reglas)

- ResultSet es un cursor:
- empieza “antes de la primera fila”
- rs.next() avanza
- Acceso a columnas:
- por nombre (más legible) o por índice (más frágil)
- Patrón de lectura:

```
while (rs.next()) { ... }
```

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.1 Recuperación: executeQuery() + ResultSet (reglas)

mapeo `ResultSet` → `objeto`

Ejemplo con `record`:

```
record Alumno(int id, String nombre, int edad) {}
```

Patrón típico (devuelve lista):

```
List<Alumno> out = new ArrayList<>();
String sql = "SELECT id, nombre, edad FROM alumnos WHERE edad >= ?";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 18);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            out.add(new Alumno(
                rs.getInt("id"),
                rs.getString("nombre"),
                rs.getInt("edad")
            ));
        }
    }
}
```

- Esta es la esencia de JDBC “bien hecho” en aplicaciones.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.2 Actualización: UPDATE con PreparedStatement

```
String sql = "UPDATE alumnos SET edad = ? WHERE id = ?";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 20);
    ps.setInt(2, 7);

    int filas = ps.executeUpdate();
    // filas==1: OK; filas==0: no existia
}
```

En JDBC es obligatorio comprobar filas cuando la lógica lo requiere.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.3 Adición: INSERT con PreparedStatement

```
String sql = "INSERT INTO alumnos(nombre, edad) VALUES (?, ?)";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, "Marta");
    ps.setInt(2, 19);

    int filas = ps.executeUpdate();
}
```

Si hay restricciones (UNIQUE/NOT NULL), aquí aparecen SQLException.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.3 Adición: INSERT con PreparedStatement

INSERT con claves generadas (autoincrement)

```
String sql = "INSERT INTO alumnos(nombre, edad) VALUES (?, ?)";

try (PreparedStatement ps =
        conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

    ps.setString(1, "Marta");
    ps.setInt(2, 19);
    ps.executeUpdate();

    try (ResultSet keys = ps.getGeneratedKeys()) {
        if (keys.next()) {
            int idNuevo = keys.getInt(1);
        }
    }
}
```

Patrón típico en DAOs: devolver el id creado.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.4 Borrado: DELETE con PreparedStatement

```
String sql = "DELETE FROM alumnos WHERE id = ?";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 7);
    int filas = ps.executeUpdate();
}
```

Puede fallar por integridad referencial (depende del esquema): se capturará como SQLException.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

JDBC imprescindible: transacciones

- Por defecto suele ser autoCommit=true:
  - cada executeUpdate() confirma automáticamente
- Para “todo o nada”:

```
conn.setAutoCommit(false);
try (PreparedStatement a = conn.prepareStatement(sql1);
     PreparedStatement b = conn.prepareStatement(sql2)) {

    a.executeUpdate();
    b.executeUpdate();
    conn.commit();

} catch (SQLException e) {
    conn.rollback();
    throw e;
} finally {
    conn.setAutoCommit(true);
}
```

- Punto crítico: **si falla una operación, rollback**.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.5 Cierre de conexiones: orden y patrón correcto

- Recursos a cerrar:
  - `ResultSet,Statement/PreparedStatement,Connection`
- Orden conceptual:
  - cerrar lo “más interno” primero (ResultSet) y lo “más externo” al final (Connection)
- Patrón recomendado:

```
try (Connection c = ...;
     PreparedStatement ps = c.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
   // ...
}
```

- Esto evita fugas y saturación del servidor.

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.5 Cierre de conexiones: orden y patrón correcto

Errores típicos por no cerrar bien

- Síntomas:
  - la app “va lenta” con el tiempo
  - “Too many connections”
  - bloqueos intermitentes
- Causa:
  - conexiones o statements abiertos
- Solución:
  - `try-with-resources` siempre
  - evitar abrir conexiones dentro de bucles si no es necesario

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.6 Excepciones: qué aporta `SQLException`

- `SQLException` no es un mensaje genérico: contiene diagnóstico útil.
- Campos relevantes:
  - `getMessage()`
  - `getSQLState()` (estándar)
  - `getErrorCode()` (motor)
  - `getNextException()` (encadenadas)

```
catch (SQLException e) {
    System.err.println(e.getMessage());
    System.err.println("SQLState=" + e.getSQLState());
    System.err.println("Code=" + e.getErrorCode());
}
```

GESTIÓN DE BBDD RELACIONALES

10.3 EJECUCIÓN DE CONSULTAS SOBRE LA BD.

10.3.6 Excepciones: qué aporta `SQLException`

Manejo correcto: rollback + propagación

- Si hay transacción:
  - en `catch`: `rollback()` antes de salir
- No hacer:
  - `catch(Exception) {}` vacío
  - seguir ejecutando como si nada
- Patrón correcto:
  - capturar, registrar, limpiar (rollback/cierre) y propagar o convertir a error de negocio.

GESTIÓN DE BBDD RELACIONALES

Apéndice (ejemplo completo) Mini-DAO: interfaz

Objetivo: aislar JDBC en una capa (DAO) y que el resto de la app no vea SQL ni ResultSet.

```
import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface AlumnoDao {
    Optional<Alumno> findById(int id) throws SQLException;
    List<Alumno> findAll() throws SQLException;
    int insert(AlumnoNuevo a) throws SQLException;   // devuelve id generado
    boolean update(Alumno a) throws SQLException;    // true si actualiza 1 fila
    boolean delete(int id) throws SQLException;      // true si borra 1 fila
}
```

Modelos (simples y claros):

```
public record Alumno(int id, String nombre, Integer edad) {}
public record AlumnoNuevo(String nombre, Integer edad) {}
```

GESTIÓN DE BBDD RELACIONALES

Apéndice (ejemplo completo) Mini-DAO: interfaz

implementación JDBC (PreparedStatement + mapeo)

```
import java.sql.*;
import java.util.*;

public class AlumnoDaoJdbc implements AlumnoDao
{
    private final String url;
    private final String user;
    private final String pass;

    public AlumnoDaoJdbc(String url, String
user, String pass) {
        this.url = url;
        this.user = user;
        this.pass = pass;
    }

    private Connection open() throws
SQLException {
        return DriverManager.getConnection(url,
user, pass);
    }
```

```
private Alumno mapRow(ResultSet rs) throws SQLException {
        int id = rs.getInt("id");
        String nombre = rs.getString("nombre");

        int edadInt = rs.getInt("edad");
        Integer edad = rs.wasNull() ? null : edadInt;

        return new Alumno(id, nombre, edad);
    }

    @Override
    public Optional<Alumno> findById(int id) throws SQLException {
        String sql = "SELECT id, nombre, edad FROM alumnos WHERE id = ?";
        try (Connection c = open();
             PreparedStatement ps = c.prepareStatement(sql)) {

            ps.setInt(1, id);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return Optional.of(mapRow(rs));
                return Optional.empty();
            }
        }
    }
```

GESTIÓN DE BBDD RELACIONALES

Apéndice (ejemplo completo) Mini-DAO: interfaz

implementación JDBC (PreparedStatement + mapeo)

```
    @Override
    public List<Alumno> findAll() throws SQLException {
        String sql = "SELECT id, nombre, edad FROM alumnos ORDER BY nombre";
        List<Alumno> out = new ArrayList<>();

        try (Connection c = open();
             PreparedStatement ps = c.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) out.add(mapRow(rs));
        }
        return out;
    }

    @Override
    public int insert(AlumnoNuevo a) throws SQLException {
        String sql = "INSERT INTO alumnos(nombre, edad) VALUES (?, ?)";
        try (Connection c = open();
             PreparedStatement ps = c.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

            ps.setString(1, a.nombre());
            if (a.edad() == null) ps.setNull(2, Types.INTEGER);
            else ps.setInt(2, a.edad());

            ps.executeUpdate();

            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) return keys.getInt(1);
                throw new SQLException("No se generó clave al insertar alumno.");
            }
        }
    }
```

GESTIÓN DE BBDD

RELACIONALES

Apéndice (ejemplo completo) Mini-DAO: interfaz

implementación JDBC (PreparedStatement + mapeo)

```
@Override
public boolean update(Alumno a) throws SQLException {
    String sql = "UPDATE alumnos SET nombre = ?, edad = ? WHERE id = ?";
    try (Connection c = open();
         PreparedStatement ps = c.prepareStatement(sql)) {

        ps.setString(1, a.nombre());
        if (a.edad() == null) ps.setNull(2, Types.INTEGER);
        else ps.setInt(2, a.edad());
        ps.setInt(3, a.id());

        return ps.executeUpdate() == 1;
    }
}

@Override
public boolean delete(int id) throws SQLException {
    String sql = "DELETE FROM alumnos WHERE id = ?";
    try (Connection c = open();
         PreparedStatement ps = c.prepareStatement(sql)) {

        ps.setInt(1, id);
        return ps.executeUpdate() == 1;
    }
}

}
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Creación de BD en MySQL (SQL)

Para crear una base de datos, necesitas una conexión al servidor de la base de datos principal (como mysql en MySQL o postgres en PostgreSQL). Desde ahí puedes ejecutar el comando CREATE DATABASE.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class CreateDatabaseExample {
    public static void main(String[] args) {
        // Conecta al servidor, no a una base de datos específica
        String url = "jdbc:mysql://localhost:3306/";
        String user = "root";
        String password = "1234";
        String sql = "CREATE DATABASE miNuevaBaseDeDatos";
                try (Connection conn = DriverManager.getConnection(url, user, password);
                     Statement stmt = conn.createStatement()) {
                    stmt.executeUpdate(sql);
                    System.out.println("Base de datos creada correctamente.");
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Creación de tabla en MySQL (SQL)

Usa sentencias SQL ejecutadas desde el código Java mediante **Statement** o **PreparedStatement.**

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class CreateTableExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/miBaseDeDatos";
        String user = "root";
        String password = "1234";

        String sql = "CREATE TABLE IF NOT EXISTS alumnos (" +
                     "id INT AUTO_INCREMENT PRIMARY KEY," +
                     "nombre VARCHAR(100)," +
                     "edad INT)";
        try (Connection conn = DriverManager.getConnection(url, user, password)){
            Statement stmt = conn.createStatement()
            stmt.executeUpdate(sql);
            System.out.println("Tabla creada");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

10.2.2 Creación de vistas en MySQL (SQL)

```
String sql = "CREATE VIEW vista_alumnos_mayores AS " +
             "SELECT nombre, edad FROM alumnos WHERE edad >= 18";
stmt.executeUpdate(sql);
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

Para operar con una base de datos ejecutando las consultas necesarias, nuestra aplicación deberá hacer las operaciones siguientes:

- **Cargar el conector** necesario para comprender el protocolo que usa la base de datos en cuestión.
- **Establecer una conexión** con la base de datos.
- **Enviar consultas** SQL y procesar el resultado.
- **Liberar los recursos** al terminar.
- **Gestionar los errores** que se puedan producir.

Podemos utilizar los siguientes tipos de sentencias:

- ***Statement***: para sentencias sencillas en SQL.
- ***PreparedStatement***: para consultas preparadas, como por ejemplo las que tienen parámetros.
- ***CallableStatement***: para ejecutar procedimientos almacenados en la base de datos.

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

Usa Statement para consultas simples y PreparedStatement para consultas parametrizadas.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class QueryExample {
    public static void main(String[] args) {
        String url =
"jdbc:mysql://localhost:3306/miBaseDeDatos";
        String user = "root";
        String password = "1234";

        String sql = "SELECT * FROM alumnos";
```

```
 try (Connection conn =
DriverManager.getConnection(url, user,
password);
             Statement stmt =
conn.createStatement();
             ResultSet rs =
stmt.executeQuery(sql)) {

            while (rs.next()) {
                int id = rs.getInt("id");
                String nombre =
rs.getString("nombre");
                int edad = rs.getInt("edad");
                System.out.println("ID: " +
id + ", Nombre: " + nombre + ", Edad: " +
edad);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

EJEMPLO PRÁCTICO:

Tienes una base de datos llamada empresa.
Una tabla llamada empleados con las columnas:

```
id (INT)
nombre (VARCHAR)
salario (DECIMAL)
```

Deseas calcular un bono del 10% sobre el salario y guardar este valor en otra tabla llamada bonos con las columnas:

```
id_empleado (INT)
nombre_empleado (VARCHAR)
bono (DECIMAL)
```

Este código realiza lo siguiente:

1. Conecta a la base de datos.
2. Recupera datos de la tabla empleados.
3. Calcula el bono para cada empleado.
4. Inserta el resultado en la tabla bonos.

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

EJEMPLO PRÁCTICO:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample {
    public static void main(String[] args) {
        // Datos de conexión
        String url =
        "jdbc:mysql://localhost:3306/empresa";
        String user = "root";
        String password = "1234";

 // Consulta para obtener datos de "empleados"
        String selectQuery =
          "SELECT id, nombre, salario FROM empleados";
```

```
// Variables para almacenar datos
y trabajar con ellos
     int idEmpleado;
     String nombreEmpleado;
     double salario;
     double bono;

// Conexión y declaración
     try (
      Connection conn =DriverManager.
         getConnection(url, user, password);
          Statement stmt =
                   conn.createStatement()) {

         // Ejecutar consulta SELECT
         ResultSet rs =
              stmt.executeQuery(selectQuery);
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

EJEMPLO PRÁCTICO:

```
// Procesar cada registro
while (rs.next()) {
  // Recuperar datos de la tabla "empleados"
       idEmpleado = rs.getInt("id");
       nombreEmpleado = rs.getString("nombre");
       salario = rs.getDouble("salario");

    // Calcular el bono (10% del salario)
       bono = salario * 0.10;

    // Mostrar en consola (opcional)
    System.out.println(
          "  ID: " + idEmpleado +
          ", Nombre: " + nombreEmpleado +
          ", Salario: " + salario +
          ", Bono: " + bono);
```

```
  //Insertar los datos en la tabla "bonos"
    String insertQuery =
     "INSERT INTO bonos (id_empleado,
      nombre_empleado, bono) " +
      "VALUES (" + idEmpleado + ", '" +
       nombreEmpleado + "', " + bono +
      ")";
    stmt.executeUpdate(insertQuery);

  } //cerramos el while
} catch (SQLException e) {
            e.printStackTrace();
   }
}
}
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

Ejemplo completo para realizar las siguientes operaciones con JDBC:

1. **Añadir registros.**
2. **Modificar registros existentes.**
3. **Recuperar datos utilizando métodos como getString, getInt, etc..**

Trabajaremos con una base de datos llamada empresa y una tabla llamada empleados con las siguientes columnas:

```
id (INT, PRIMARY KEY, AUTO_INCREMENT)
nombre (VARCHAR)
edad (INT)
salario (DECIMAL)
```

Este código realiza:

1. Inserción de registros (INSERT).
2. Modificación de registros (UPDATE).
3. Recuperación de datos (SELECT).
4. Uso de métodos como getString, getInt.

GESTIÓN DE BBDD RELACIONALES

INTRODUCCIÓN

OTROS EJEMPLOS

Realizar Consultas.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCOperations {
    public static void main(String[] args) {
        // Datos de conexión
        String url = "jdbc:mysql://localhost:3306/empresa";
        String user = "root";
        String password = "1234";

        try (Connection conn = DriverManager.getConnection(url, user, password);
             Statement stmt = conn.createStatement()) {

           // 1. Añadir registros a la tabla "empleados"
                        String insertQuery = "INSERT INTO empleados (nombre, edad,
           salario) VALUES " +
                                            "('Juan', 30, 3000.00), " +
                                            "('Ana', 25, 3500.00), " +
                                            "('Luis', 28, 2800.00)";
                        int rowsInserted = stmt.executeUpdate(insertQuery);
                        System.out.println(rowsInserted + " registros insertados.");
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

```
// 2. Modificar un registro existente
            String updateQuery =
                 "UPDATE empleados SET salario = salario + 500 WHERE nombre = 'Juan'";
            int rowsUpdated = stmt.executeUpdate(updateQuery);
            System.out.println(rowsUpdated + " registros actualizados.");

 // 3. Recuperar datos de la tabla "empleados"
            String selectQuery = "SELECT id, nombre, edad, salario FROM empleados";
            ResultSet rs = stmt.executeQuery(selectQuery);

 // Mostrar los datos usando métodos getString, getInt, etc.
             System.out.println("Datos de la tabla empleados:");
             while (rs.next()) {
                 int id = rs.getInt("id");
                 String nombre = rs.getString("nombre");
                 int edad = rs.getInt("edad");
                 double salario = rs.getDouble("salario");

                 System.out.println("ID: " + id + ", Nombre: " + nombre +
                                    ", Edad: " + edad + ", Salario: " + salario);
             }

        } catch (SQLException e) {
                e.printStackTrace();
    } } }
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

Manejar múltiples conexiones

Puedes abrir múltiples conexiones llamando **a DriverManager.getConnection** varias veces.

Si trabajas con muchas conexiones, considera usar **un pool de conexiones** (como HikariCP).

```
Connection conn1 = DriverManager.getConnection("jdbc:mysql://localhost:3306/miBase1",
"user1", "password1");
Connection conn2 = DriverManager.getConnection("jdbc:mysql://localhost:3306/miBase2",
"user2", "password2");

// Usa ambas conexiones y ciérralas al terminar
conn1.close();
conn2.close();
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

Manejar múltiples conexiones

Configurar un pool de conexiones con Apache Commons DBCP

```
import
org.apache.commons.dbcp2.BasicDataSource;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class DBCPExample {
  private static DataSource dataSource;
```

```
  // Configuración inicial del pool de conexiones
static {
  BasicDataSource ds = new BasicDataSource();
  ds.setUrl("jdbc:mysql://localhost:3306/miBaseDeDatos");
  ds.setUsername("root");
  ds.setPassword("1234");
  // Configuración opcional
  ds.setMinIdle(2);   //Mínimo de conexiones inactivas
  ds.setMaxIdle(5);   //Máximo de conexiones inactivas
  ds.setMaxTotal(10); //Máximo de conexiones totales
  ds.setInitialSize(3); //Conexiones creadas al inicio
  ds.setMaxWaitMillis(10000); //Tiempo máximo para esperar
  dataSource = ds;
}
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Realizar Consultas.

Manejar múltiples conexiones

Configurar un pool de conexiones con Apache Commons DBCP

```
 public static void main(String[] args) {
        // Obtener una conexión del pool y ejecutar una consulta
        try (Connection conn = dataSource.getConnection()) {
            String sql = "SELECT * FROM alumnos";
            try (PreparedStatement pstmt = conn.prepareStatement(sql);
                 ResultSet rs = pstmt.executeQuery()) {

                while (rs.next()) {
                    System.out.println("ID: " + rs.getInt("id") + ", Nombre: "
+ rs.getString("nombre"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

GESTIÓN DE BBDD RELACIONALES

OTROS EJEMPLOS

Ejercicio Propuestos.

Desarrolla una aplicación en Java que realice lo siguiente utilizando JDBC para conectarse a un servidor MySQL:

Crear una **base de datos** llamada **tienda**.

Crear una **tabla** en la base de datos llamada productos con las siguientes columnas:

id (clave primaria, entero, autoincremental).

nombre (cadena de texto, no nula).

categoria (cadena de texto, no nula).

precio (decimal, no nulo).

Insertar varios registros de ejemplo en la tabla productos.

Consultar los productos con un precio mayor a un valor proporcionado por el usuario y mostrar los resultados.

Permitir al usuario eliminar un producto especificando su ID.

TEMA 10: GESTIÓN DE BASES DE DATOS RELACIONALES